

# Specification: Strategies for the automated identification of service interfaces during the migration from monoliths to microservices

1<sup>st</sup> Alexander Ley

Computer Science Department  
University of applied science Bonn-Rhein-Sieg  
Sankt Augustin, Germany  
s6alleyy@outlook.de

## I. CONTEXT AND MOTIVATION

In Software Engineering of business applications, the architecture and organization of the software components plays a pivotal rule to enable desired features like scalability, maintainability and deployability.

As software systems become more and more complex these features are more and more less achieved due to the predominant implementation of *Monolithic Architectures* [1].

According to Lewis and Fowler [2] a monolith is an application built as a single unit. Enterprise applications are traditionally divided into three parts:

- 1) A client-side user interface
- 2) A Server-side application incorporating the business long-distance
- 3) A database

Monolithic Architectures decompose the software according these above presented horizontal, technical oriented layers. Between these layers are lots of dependencies resulting in a loss of flexibility whenever the software have to be adapted hindering agile software development and maintenance. As shown in figure 1 the deployability and scalability of monoliths is limited as always only the application as whole can be scaled. [1]

As the requirements towards the software are changing and evolving there is need for software to be locally and flexible adaptable and deployable without influencing other parts of the software.

Because of the downsides of monolithic architectures researches has reached out to other software architecture paradigm [3]. *Microservice Architectures* seeks to overcome the shortcomings of monolithic architectures and have therefore gained significant attraction [4]. The core idea behind this novel kind of software organization is to decompose the software vertically according to the enterprise business logic rather than dividing technically.

Lewis and Fowler [2] described *Microservice Architectures*:

“The microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and

communicating with lightweight mechanisms, often an HTTP resource API [today often a RESTful API [4]]. These services are built around business capabilities and independently deployable by fully automated deployment machinery. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies.” [2]

Microservices are typically build in a way that they encapsulate one single business responsibility which can encompass an own database, own User-Interface and business logic seeking to being independent to other Microservices. Because of their independently they can maintained, adapted and deployed individually which fits more to companies desire for agile software development. [1]

Therefore many companies which have invested into the development of a monolithic enterprise application may consider to migrate to a microservice architecture. The migration is a complex and non-trivial process. The main challenge in the migration process is the identification of microservice.

One method is to hire a migration engineer to analyze and identify the application’s domain to identify microservices candidates using e.g. techniques like *Domain-Driven Design (DDD)* [1]. DDD is a collection of abstract concepts helping to model complex and large software with respect to expertise [5].

Other more automatic techniques being less costly involve static code analysis where tools usually represent the code as graphs looking for highly coupled nodes which may build a microservice. Graphs can be build on focusing on different features of the code like the usage of classes. A more dynamic approach is to look during the execution of the software on the traces and data flows to identify service cuts. [1]

## II. PROBLEMS AND QUESTIONS

The decomposition of a monolith into microservices is a complicate and expensive task and is still at early stage, because of the following reasons:

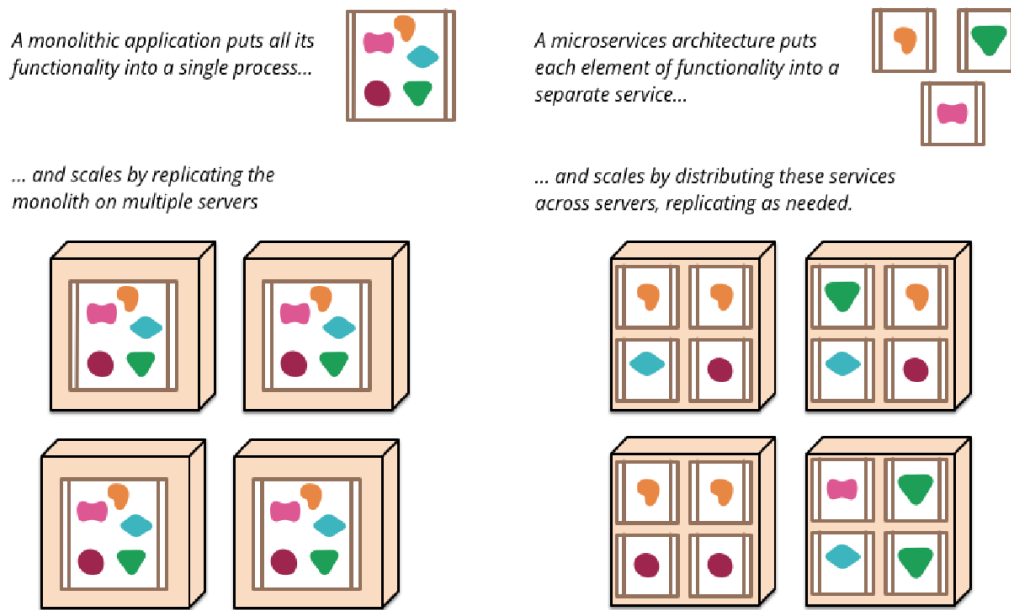


Fig. 1. Monoliths and Microservices by Lewis and Fowler

- lack of integrated, comprehensive data collection and analysis methods with regard to crucial aspects of the monolithic application
- lack of comparison between various decomposition methods and algorithms
- lack of validated and widely accepted metrics and benchmarks to measure the quality of resulting microservice candidates
- shortage of integrated tools to support the various stages of the microservice extraction pipeline

Furthermore existing material tends to focus on specific scenarios, domains or programming languages.

Therefore the following questions arise:

- How can a monolithic application automatically being divided into microservices?
  - How to recognize the service-interfaces?
- Which tools exists? How well do they perform?
- What is the current state of literature according to the question above?

### III. METHODS

To answer the above question the planned work seeks to perform a *Systematic Literature Review (SLR)*. The reader is expected to gain insight into the various existing tools, a comparison of them and which are currently prevailing. Furthermore, the work will shed light and discuss the challenge, identify research gaps and give an overview of the current state of research.

### REFERENCES

- [1] Y. Abgaz, A. McCarren, P. Elger, D. Solan, N. Lapuz, M. Bivol, G. Jackson, M. Yilmaz, J. Buckley, and P. Clarke, "Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review," vol. 49, no. 8, pp. 4213–4242. [Online]. Available: <https://ieeexplore.ieee.org/document/10160171/>
- [2] J. Lewis and M. Fowler, "Microservices." [Online]. Available: <https://martinfowler.com/articles/microservices.html>
- [3] M. Daoud, A. E. Mezouari, N. Faci, D. Benslimane, Z. Maamar, and A. E. Fazziki, "Automatic Microservices Identification from a Set of Business Processes," in *Smart Applications and Data Analysis*, M. Hamlich, L. Bellatreche, A. Mondal, and C. Ordonez, Eds. Springer International Publishing, vol. 1207, pp. 299–315. [Online]. Available: [http://link.springer.com/10.1007/978-3-030-45183-7\\_23](http://link.springer.com/10.1007/978-3-030-45183-7_23)
- [4] M. Garriga, "Towards a Taxonomy of Microservices Architectures," in *Software Engineering and Formal Methods*, A. Cerone and M. Roveri, Eds. Springer International Publishing, vol. 10729, pp. 203–218. [Online]. Available: [http://link.springer.com/10.1007/978-3-319-74781-1\\_15](http://link.springer.com/10.1007/978-3-319-74781-1_15)
- [5] C. Nockemann, "Domain-Driven Design in der Praxis."
- [6] M. Abdellatif, A. Shatnawi, H. Mili, N. Moha, G. E. Boussaidi, G. Hecht, J. Privat, and Y.-G. Guéhéneuc, "A taxonomy of service identification approaches for legacy software systems modernization," vol. 173, p. 110868. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0164121220302582>
- [7] J. Fritsch, J. Bogner, A. Zimmermann, and S. Wagner, "From Monolith to Microservices: A Classification of Refactoring Approaches," in *Software Engineering Aspects of Continuous Development and New Paradigms*

*of Software Production and Deployment*, J.-M. Bruel, M. Mazzara, and B. Meyer, Eds. Springer International Publishing, vol. 11350, pp. 128–141. [Online]. Available: [http://link.springer.com/10.1007/978-3-030-06019-0\\_10](http://link.springer.com/10.1007/978-3-030-06019-0_10)

- [8] F. Ponce, G. Marquez, and H. Astudillo, “Migrating from monolithic architecture to microservices: A Rapid Review,” in *2019 38th International Conference of the Chilean Computer Science Society (SCCC)*. IEEE, pp. 1–7. [Online]. Available: <https://ieeexplore.ieee.org/document/8966423/>
- [9] V. Velepucha and P. Flores, “Monoliths to microservices - Migration Problems and Challenges: A SMS,” in *2021 Second International Conference on Information Systems and Software Technologies (ICI2ST)*. IEEE, pp. 135–142. [Online]. Available: <https://ieeexplore.ieee.org/document/9447352/>